Polynomial Techniques

NTT applications, inverse / log / exp, Berlekamp–Massey, Lagrange interpolation

Learning goals

- Treat sequences as **formal power series** and manipulate them in O(n log n)
- Derive inverse, log and exp with **Newton's method** and implement them correctly
- Use log/exp to count connected structures and sets of structures
- Find a linear recurrence from terms with **Berlekamp–Massey**, then jump to term 10^{18}
- Evaluate a polynomial from samples with **Lagrange interpolation** in O(n)

Motivation

Count connected labelled graphs on k vertices, for every $k \le 10^5$, mod 998244353.

- All graphs on k vertices: g_k = 2^{k(k−1)/2}, easy
- Every graph = **set** of connected components
- Exponential generating functions: **G(x) = exp(C(x))**
- So **C(x) = ln G(x)**, one O(n log n) call

```
k :  1  2  3   4    5      6
c_k: 1  1  4  38  728  26704
```

NTT recap (nationals-07)

- $p = 998244353 = 119 \cdot 2^{23} + 1$, primitive root 3
- Length-n transform exists for $n \mid 2^{23}$: evaluate at n-th roots of unity
- Multiply: pad to power of two $\geq |a| + |b| - 1$, transform, pointwise, inverse
- Inverse transform = forward transform, reverse $a[1..n-1]$, divide by n

Product of two length-2^{18} inputs: 3 transforms of length 2^{19}, ≈ 60 ms with the reference code.

NTT applications catalogue

Problem shape	Tool
count pairs with a_i + b_j = s	convolve frequency arrays
match a pattern with wildcards	$\Sigma (p - t)^2 p t$ as 3 convolutions
product of many small polynomials	divide & conquer, $O(n \log^2 n)$
A(x)^k for huge k	exp(k ln A)
divisor-count convolution (WF 2015 *Tile Cutting*)	one big product
arbitrary modulus (10^9+7)	3 NTT primes + CRT, or split FFT

Product of many polynomials

$\prod (1 + a_i x)$ for $n = 10^5$ factors:

- multiplying left to right: $O(n^2)$
- **divide & conquer**: product(l, r) = product(l, m) · product(m, r)
- each level costs O(n log n) total, log n levels → **$O(n \log^2 n)$**

```
poly prod(int l, int r) {           // factors f[l..r)
    if (r - l == 1) return f[l];
    int m = (l + r) / 2;
    return multiply(prod(l, m), prod(m, r));
}
```

Formal power series

- Work in F_p[[x]]: infinite series, but we only keep **mod x^n**
- A has an inverse $\Leftrightarrow a_0 \neq 0$
- ln A defined (as a series) when $a_0 = 1$
- exp A defined when $a_0 = 0$
- Derivative and integral are coefficient-wise, O(n)
  - integral needs inverses of 1..n, so n < p

Newton's method on power series

Want B with $F(B) \equiv 0 \pmod{x^n}$. If B_0 is correct mod x^m:

```
B₁ = B₀ − F(B₀) / F'(B₀)        is correct mod x^{2m}
```

- Same quadratic convergence as real Newton: **precision doubles**
- Total cost T(n) = T(n/2) + O(M(n)) = **O(M(n)) = O(n log n)**
- Every operation today is one Newton iteration loop

Inverse: B = 1/A

$F(B) = 1/B - A$, $F'(B) = -1/B^2$

$$B \leftarrow B - (1/B - A) \,/\, (-1/B^2) = B\,(2 - A\,B) \qquad (\mathrm{mod}\ x^{2m})$$

- Start: $B = a_0^{-1}$ (correct mod x^1)
- Each step: two multiplications of size 2m
- Only the first 2m coefficients of A are needed in step m

Inverse: code

```
poly inverse(const poly& A, int n) {          // requires A[0] != 0
    poly B{(int)power(A[0], MOD - 2)};
    for (int m = 1; m < n; m <<= 1) {
        poly a(A.begin(), A.begin() + min((int)A.size(), 2 * m));
        poly ab = multiply(a, B);
        ab.resize(2 * m);
        for (int& x : ab) x = x ? MOD - x : 0;  // -A B
        ab[0] = (ab[0] + 2) % MOD;              // 2 - A B
        B = multiply(B, ab);
        B.resize(2 * m);
    }
    B.resize(n);
    return B;
}
```

Logarithm

$\ln A = \int A' / A$ (requires $a_0 = 1$, gives $(\ln A)_0 = 0$)

```
poly logarithm(const poly& A, int n) {
    poly a(A.begin(), A.begin() + min((int)A.size(), n));
    poly q = multiply(derivative(a), inverse(a, n));
    q.resize(n - 1);
    poly r = integral(q);
    r.resize(n);
    return r;
}
```

One inverse + one product: O(n log n)

Exponential

Solve ln B − A = 0 with Newton: F′(B) = 1/B

$$B \leftarrow B - (\ln B - A) \cdot B = B\,(1 - \ln B + A) \pmod{x^{2m}}$$

```cpp
poly exponent(const poly& A, int n) {          // requires A[0] == 0
    poly B{1};
    for (int m = 1; m < n; m <<= 1) {
        poly l = logarithm(B, 2 * m);
        for (int i = 0; i < 2 * m; i++)
            l[i] = ((ll)(i < (int)A.size() ? A[i] : 0) - l[i] + MOD) % MOD;
        l[0] = (l[0] + 1) % MOD;
        B = multiply(B, l);
        B.resize(2 * m);
    }
    B.resize(n);
    return B;
}
```

Cost in practice

Operation	Complexity	$n = 2^{18}$ (reference code, -O2)
multiply	O(n log n)	~60 ms
inverse	O(n log n), a few M(n)	~160 ms
log	O(n log n)	~210 ms
exp	O(n log n), large constant	~420 ms

- exp is the slow one: precompute, avoid calling it inside loops
- Faster exp exists (semi-relaxed / CDQ convolution) if limits are tight

Why log and exp count things

Labelled structures, EGFs: A(x) = Σ a_k x^k / k!

- **Sets** of connected objects: Total = **exp**(Connected)
- So Connected = **ln**(Total)
- Sequences of objects: 1 / (1 − A)
- Powers: A^k = exp(k · ln A), valid for huge k when $a_0 = 1$

Unlabelled versions use the Euler transform: exp(Σ_j A(x^j)/j). Same tools.

Worked example: connected labelled graphs

```
poly G(n + 1);
for (int k = 0; k <= n; k++)                          // EGF coefficient 2^{C(k,2)} / k!
    G[k] = power(2, (ll)k * (k - 1) / 2 % (MOD - 1)) * ifact[k] % MOD;
poly C = logarithm(G, n + 1);
for (int k = 1; k <= n; k++) answer[k] = C[k] * fact[k] % MOD;
```

- Exponent k(k−1)/2 reduced mod p − 1 (Fermat): reduce exponents mod p − 1, never mod p
- Output: 1, 1, 4, 38, 728, 26704, … ✔

Linear recurrences: the k-th term

$s_i = c_1 s_{i-1} + \ldots + c_L s_{i-L}$

- Matrix power: $O(L^3 \log k)$, fine for $L \le 100$
- **Kitamasa**: $s_k = \sum r_j s_j$ where $r(x) = x^k \bmod \text{charpoly}$
  - square-and-multiply on polynomials mod charpoly: **$O(L^2 \log k)$**
  - with NTT polynomial mod: $O(L \log L \log k)$
- But where do the c_j come from?

Berlekamp–Massey

Input: 2L terms of a sequence satisfying *some* order-L recurrence (over a field)
Output: the **shortest** recurrence generating them, in $O(n^2)$

Idea, processing terms one by one:

- keep current recurrence C and the last one B that failed
- at term i, compute discrepancy d = s_i + Σ C_j s_{i−j}
- d = 0: C still works
- d ≠ 0: fix C by subtracting (d / b) · x^m · B, and swap if the length must grow

Berlekamp–Massey: code

```
vector<ll> berlekampMassey(const vector<ll>& s) {
    int n = s.size(), L = 0, m = 0;
    vector<ll> C(n + 1), B(n + 1), T;
    C[0] = B[0] = 1;
    ll b = 1;
    for (int i = 0; i < n; i++) {
        m++;
        ll d = s[i] % MOD;
        for (int j = 1; j <= L; j++) d = (d + C[j] * s[i - j]) % MOD;
        if (d == 0) continue;
        T = C;
        ll coef = d * power(b, MOD - 2) % MOD;
        for (int j = m; j <= n; j++) C[j] = (C[j] - coef * B[j - m] % MOD + MOD) % MOD;
        if (2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }
    vector<ll> res(L);
    for (int j = 0; j < L; j++) res[j] = (MOD - C[j + 1]) % MOD;
    return res;                          // s[i] = sum res[j] * s[i-1-j]
}
```

The BM workflow at contests

1. Suspect the answer satisfies a linear recurrence (DP with **fixed-size** linear transition, counting tilings of k × n strips, walks in a fixed graph ...)
2. Write the slow DP, print the first ~2L + 10 terms (be generous: 500–1000)
3. BM → recurrence; check it predicts the extra terms
4. Kitamasa → term n up to 10^{18}

Requires a **prime** modulus (field). For mod 2^{32} or composite mods use other methods.

Lagrange interpolation

A degree-d polynomial is determined by d + 1 samples:

$P(x) = \Sigma_i\, y_i \cdot \Pi_{j \neq i}\, (x - x_j) / (x_i - x_j)$

- General points: $O(d^2)$
- **Consecutive points x_i = 0..d**: denominators are ± i! (d − i)!
  - prefix/suffix products of (x − j) → **O(d)**
- Typical source: $\Sigma_{i \leq n}\, i^k$ is a polynomial of degree k + 1 in n

Lagrange: O(d) evaluation

```
ll lagrangeConsecutive(const vector<ll>& y, ll x) {   // y[i] = P(i), i = 0..d
    int d = (int)y.size() - 1;
    if (x <= d) return y[x];
    vector<ll> pre(d + 2, 1), suf(d + 2, 1), ifact(d + 1, 1);
    for (int i = 0; i <= d; i++) pre[i + 1] = pre[i] * ((x - i) % MOD) % MOD;
    for (int i = d; i >= 0; i--) suf[i] = suf[i + 1] * ((x - i) % MOD) % MOD;
    ll f = 1; for (int i = 1; i <= d; i++) f = f * i % MOD;
    ifact[d] = power(f, MOD - 2);
    for (int i = d; i > 0; i--) ifact[i - 1] = ifact[i] * i % MOD;
    ll ans = 0;
    for (int i = 0; i <= d; i++) {
        ll t = y[i] * pre[i] % MOD * suf[i + 1] % MOD * ifact[i] % MOD * ifact[d - i] % MOD;
        ans = ((d - i) & 1) ? (ans - t + MOD) % MOD : (ans + t) % MOD;
    }
    return ans;
}
```

Codeforces 622F: $n \le 10^9$, $k \le 10^6$ → $O(k \log k)$.

Spotting "it's a polynomial"

- DP over n whose answer is a sum of i^k-like terms
- dp over values up to 10^9 where the transition is "sum of previous": dp is piecewise polynomial in the value → interpolate per piece
- Counting lattice points / colourings with a fixed structure as a function of n
- **Test it**: compute d + 2 samples by brute force and check that the (d+1)-th finite difference vanishes

Pitfalls

- NTT length must divide 2^{23} for 998244353; arrays beyond $8 \cdot 10^6$ need another prime
- log needs $a_0 = 1$, exp needs $a_0 = 0$: normalise first (divide by a_0, factor out x^t)
- Resizing: keep exactly 2m coefficients each Newton step, or errors leak in
- Exponents like 2^{k(k−1)/2}: reduce mod p − 1, not mod p
- BM over a non-prime modulus is **wrong** (needs division)
- Too few terms for BM: you get a short, wrong recurrence. Verify on extra terms
- Lagrange on something that is **not** a polynomial of degree ≤ d: silently wrong. Check with an extra sample

Practice problems

- Library Checker: *Inv of Formal Power Series*, *Log of …*, *Exp of …*, *Pow of …*
- Library Checker: *Find Linear Recurrence*, *Kth term of Linearly Recurrent Sequence*
- Codeforces 622F *The Sum of the k-th Powers* (Lagrange)
- Codeforces 438E *The Child and Binary Tree* (inverse + sqrt)
- Codeforces 1096G *Lucky Tickets* (NTT power)
- Codeforces 528D *Fuzzy Search* (FFT matching)
- Codeforces 1251F *Red-White Fence* (NTT products)
- WF 2015 *Tile Cutting* (Kattis)

Summary

- Power series mod x^n + NTT = a full algebra toolkit in O(n log n)
- Newton's iteration gives inverse, log, exp (and sqrt, pow) from one pattern
- log/exp translate "connected" ↔ "all" for labelled structures
- Berlekamp–Massey turns a slow DP into a recurrence; Kitamasa jumps to 10^{18}
- Lagrange interpolation evaluates polynomial-valued answers at huge n in O(d)